YELLOW PAPER

Encrypted Zero-Knowledge Payments on Stacks: A Formal Specification

John B Mukhwana

September 2026

Reference implementation	Stacks Shield (proof of concept)
Network	Stacks Testnet
Deployer (v2, current)	`ST18XMPE0PS5VNEEKB82BPW7NRZRHXEPH16JK8NN6`
Circuit version	2

Abstract

This document is the formal specification of a shielded-payment protocol for the Stacks blockchain. It is written as a parameterized construction, so it describes the full design rather than only the choices in the current build. Every component is given twice: a precise definition, and an explanation of how it is achieved in practice, including the procedures a prover, a contract, and an operator follow. The deployed proof of concept, called Stacks Shield, is presented as one instantiation, and instantiation notes mark those concrete choices.

The paper covers the note and commitment scheme, the commitment tree and how it is maintained, ownership and nullifiers, operations as a general consume-and-produce relation with the prover procedure, the canonical public-input encoding and how the components agree on it, the tree-transition binding, an abstract settlement predicate with each admissible verification backend and how it is built, the on-chain state and invariants, fees, and the security model. Equations are kept short; the prose carries the mechanism. The companion White Paper covers motivation, use cases, and roadmap.

Contents

1 Scope and Conventions

This specification is enough to reimplement any component and match the deployed system. It fixes what a proof commits to, what each contract checks, and how the off-chain and on-chain parts agree. It does not fix the wire formats of the API or the relayer, which are implementation detail.

Three roles appear throughout. The *prover* is the user's client, which holds secrets and builds proofs. The *contracts* are the on-chain programs, which hold no secrets and check proofs and accounting. An *operator* is an off-chain party that verifies proofs or relays transactions. The protocol is designed so that no operator can move funds or forge an operation; the trust that does exist is stated explicitly in Section 10.

A value written in a fixed-width font, such as `new_root`, names a concrete field in the implementation, so the definition here and the source can be checked against each other.

2 Notation

The symbols used throughout are collected here for reference. Each is defined in full where it first appears.

Symbol	Meaning
$\mathbb{F}$	the prime field over which the circuits operate
H_2, H_4	Poseidon hash on two and four field elements
keccak256	Keccak-256, the statement hash over serialized field elements
E, G	the embedded curve and its generator, used for ownership
Π	the proof system (prover and verifier)
d, EMPTY	tree depth and the empty-leaf value
v	the protocol version tag bound into every proof
$n = (a, pk, r)$	a note: amount, owner public key, blinding
$pk = (pk_X, pk_Y)$, sk	owner public key and its secret scalar, with $[sk]G = pk$
α	asset identifier for a multi-asset deployment
C, oc	note commitment and owner commitment
N	nullifier of a spent note
i, $\mathbf{b}$, $\mathbf{s}$	leaf index, its bit decomposition, and its sibling path
R, R_{old}, R_{new}	a tree root, and the roots before and after an insertion
$\mathscr{I}$, $\mathscr{O}$	the input and output note multisets of an operation
$\mathscr{N}$	the on-chain append-only nullifier set
h	the statement digest of an operation
ρ	an aggregation root published on chain
Accept	the settlement predicate on a statement digest
feUint, fe	field encodings of an integer and of a principal
M, N	threshold parameters of the federated backend

3 Parameters

The protocol is defined over the parameters below. Any choice meeting the stated properties is admissible, which is what lets the same construction move from the current backend to a fully native one without touching the rest of the system.

- A prime field $\mathbb{F}$ over which the circuits operate. All in-circuit values are elements of $\mathbb{F}$, and everything hashed on chain is a 32-byte field element.
- A hash family $H_k : \mathbb{F}^k \to \mathbb{F}$ that is collision-resistant and hiding. Collision resistance gives binding for commitments and soundness for the tree; hiding keeps a commitment from leaking its note.
- An embedded curve $E/\mathbb{F}$ with generator G. Embedded means the curve's coordinates are elements of $\mathbb{F}$, so a scalar multiplication can be computed inside the circuit cheaply. This is what makes ownership provable in zero knowledge.
- A proof system Π with a prover and a verifier for the circuit relations.
- A byte-level hash keccak256 used once per operation to compress the public inputs into a single statement digest.
- A tree depth d, giving capacity 2^d leaves, and an empty-leaf value EMPTY.
- A version tag v bound into every proof, so a proof for one protocol version can never be replayed against another.

Instantiation. $\mathbb{F}$ is the BN254 scalar field; H_k is Poseidon; E is Grumpkin; Π is UltraHonk (Barretenberg); keccak256 is Keccak-256; $d = 20$; $\mathsf{EMPTY} = 0$; $v = 2$. Full constants are in Appendix A.

Operation codes are SHIELD $= 1$, TRANSFER $= 2$, WITHDRAW $= 3$, SPLIT $= 4$, MERGE $= 5$. The code space is open, so a new operation is added by assigning a new code and a new circuit (Section 7).

4 Notes and Commitments

4.1 The note

Value is held in notes rather than in account balances. A note is private data known only to its owner. On chain it appears only as a commitment, and nothing else about it is ever published in the clear.

Definition 4.1 (Note)**.** A note is $n = (a, pk, r)$ with amount a, owner public key $pk = (pk_X, pk_Y) \in \mathbb{F}^2$, and blinding $r \in \mathbb{F}$. For a multi-asset deployment it also carries an asset identifier $\alpha \in \mathbb{F}$.

The blinding is the reason two notes of the same amount are indistinguishable. It is sampled uniformly at note creation and never reused. Sampling below the field modulus matters: a value at or above $|\mathbb{F}|$ is not a field element and would be reduced, which could bias the distribution, so the client draws a value strictly below the modulus and nonzero.

Instantiation. r is drawn as 31 random bytes (hence below $2^{248} < |\mathbb{F}|$) plus one, giving $r \in [1, 2^{248}]$. Amounts satisfy $0 \le a < 2^{64}$, enforced in every circuit by a range check.

4.2 Commitment

Definition 4.2 (Commitment)**.** The commitment of a note is a hiding, binding hash

$$C \;=\; H\big(a,\, pk_X,\, pk_Y,\, r;\; \alpha\big).$$

Binding means no one can find a second note with the same commitment, so a spender cannot later claim a different amount or owner for it; this follows from collision resistance of H. Hiding means the commitment reveals none of a, pk, or r; this follows from the blinding r being secret and uniform. The chain stores C and learns nothing more.

Instantiation. Native STX uses $C = H_4(a, pk_X, pk_Y, r)$. A SIP-10 asset uses the nested form $C = H_2\big(H_4(a, pk_X, pk_Y, r),\, \alpha\big)$, so the same four-input inner hash is reused and the asset is folded in by one extra hash. This is `commitmentOf` in the client and the commitment gadget in the circuit.

4.3 Asset binding

Definition 4.3 (Asset identifier)**.** An asset deployed at principal A has identifier $\alpha = \mathrm{fe}(A) \in \mathbb{F}$, where fe is any injective, collision-resistant reduction of the asset's on-chain identity to a field element.

Folding α into the commitment is what keeps assets separate in one shared pool. Since C, and therefore the nullifier and the tree membership derived from it, are all functions of α, a proof built for one asset fixes values that cannot satisfy the relation for another. No separate per-asset tree is needed, so the anonymity set stays unified while the accounting stays exact (Section 11).

Instantiation. $\mathrm{fe}(A) = \mathrm{sha256}(\sigma(A))$ with the top byte cleared, where $\sigma(A)$ is the Clarity consensus serialization of the principal. Clearing the top byte bounds the result below $2^{248} < |\mathbb{F}|$, so it is always a valid field element, and matches `fe-principal` on chain.

4.4 Owner commitment

Alongside the note commitment, leaf-adding operations publish an owner commitment $oc = H(pk_X, pk_Y)$. It is asset-agnostic and lets the protocol record who a new note belongs to, in committed form, without revealing the key. It is a public input so the contract can bind it into the statement.

5 The Commitment Tree

5.1 Structure and maintenance

All commitments, across all assets, are the leaves of a single append-only Merkle tree of depth d under leaf hash H_2. Leaves are filled left to right. The next free index is the current leaf count, and once a leaf is written it never changes.

A full tree of depth 20 has over a million nodes, so the registry does not store it whole. It keeps the current root and enough frontier state to append the next leaf and to serve membership witnesses. The client mirrors the same structure from the public commitment stream so it can build proofs offline. Both sides use the identical hash and ordering, so they always agree on the root.

5.2 Root computation

Definition 5.1 (Root fold)**.** For a leaf ℓ, the index bits $\mathbf{b} = (b_0, \ldots, b_{d-1})$ of an index i (with $i = \sum_j b_j 2^j$), and siblings $\mathbf{s} \in \mathbb{F}^d$, define

$$x_0 = \ell, \qquad x_{j+1} = \begin{cases} H_2(s_j, x_j) & b_j = 1 \\ H_2(x_j, s_j) & b_j = 0 \end{cases}, \qquad \text{root}(\ell, \mathbf{b}, \mathbf{s}) = x_d.$$

The fold walks from the leaf to the root, one level per step. At each level the bit b_j says whether the running node is the right child (so the sibling is on the left) or the left child. Membership of ℓ under a root R at index i is simply $\text{root}(\ell, \mathbf{b}(i), \mathbf{s}) = R$. A spend proves this equation in zero knowledge, so it shows the input note is in the tree without revealing which leaf it is.

Instantiation. This is `compute_merkle_root` in the circuit library; the bit decomposition check is `assert_index_bits`.

5.3 Insertion witness

Adding a leaf must not only produce a new root, it must produce the *correct* new root. The insertion witness makes that provable.

Definition 5.2 (Insertion)**.** An insertion of leaf ℓ at empty slot i with siblings $\mathbf{s}$ is valid iff

$$\text{root}(\mathsf{EMPTY}, \mathbf{b}(i), \mathbf{s}) = R_{\text{old}} \quad \text{and} \quad \text{root}(\ell, \mathbf{b}(i), \mathbf{s}) = R_{\text{new}}.$$

The first equation proves the slot really was empty under the old root, using the same sibling path. The second proves that placing ℓ in that slot, with those same siblings, yields exactly the advertised new root. Because both use one set of siblings, they pin down a single legitimate R_{new} for the given R_{old}, ℓ, and i. Section 9 explains why this matters.

Instantiation. This is `assert_insertion`, with $\mathsf{EMPTY} = 0$.

Example 5.1 (Index bits)**.** For $i = 5$ and $d = 20$ the bits are $\mathbf{b}(5) = (1, 0, 1, 0, 0, \ldots, 0)$, since $5 = 2^0 + 2^2$. The fold goes right at levels 0 and 2 and left elsewhere.

6 Ownership and Nullifiers

6.1 Proving ownership

Definition 6.1 (Ownership)**.** A prover owns a note with public key pk iff it knows a scalar sk with $[sk]G = pk$.

Ownership is checked inside the circuit by computing the scalar multiplication $[sk]G$ on the embedded curve and asserting it equals pk. Because E is embedded in $\mathbb{F}$, this is a small, exact in-circuit computation rather than an expensive simulation. The secret never leaves the client; only the proof that the equation holds is published.

Instantiation. Owner keys are derived by a small key-generation circuit so that the public key matches the same in-circuit operation the spend circuits check. The secret is reduced to $sk \in [1, 2^{240}]$ before use.

6.2 Nullifiers

Definition 6.2 (Nullifier)**.** The nullifier of a note with commitment C owned under secret sk is $N = H_2(C, sk)$.

A nullifier is how a spend marks a note as consumed. It is deterministic, so the same note always yields the same N, which is what makes a second spend detectable. It is unlinkable, because computing N requires sk, so no observer can connect a published nullifier back to the commitment it retires. The registry keeps every N in an append-only set and rejects any repeat, which is the mechanism that prevents double-spends (Section 11). Because C is asset-bound, N is too.

7 Operations

7.1 The general relation

Definition 7.1 (Operation)**.** An operation consumes a multiset of input notes $\mathscr{I}$ and produces a multiset of output notes $\mathscr{O}$ of a single asset, subject to conservation

$$\sum_{n \in \mathscr{I}} n.a \;=\; \sum_{n \in \mathscr{O}} n.a \;+\; \text{fee}.$$

The proof for an operation asserts, for each input, that the prover owns it, that its commitment is a member of the tree under a root R, and that its nullifier is correctly formed; for each output, that its commitment and owner commitment are well-formed; the version tag v; and a 64-bit range check on every output amount so no amount can wrap the field or go negative. If the operation adds leaves it also binds the tree transition (Section 9).

7.2 How a proof is built

For a spend the prover follows one procedure, regardless of which of the concrete operations it is:

1. Select the input notes and read their secrets and blindings.
2. Recompute each input commitment and locate it in the mirrored tree to obtain its index and sibling path under the current root R.
3. Derive each nullifier $N = H_2(C, sk)$.
4. Construct the output notes with fresh blindings and their commitments and owner commitments.
5. For a leaf-adding operation, compute the insertion witness for each new leaf at the next free slot, giving R_{new} and the index bits and siblings.
6. Assemble the public inputs (Section 8) and the private witness, and run Π to produce the proof.

The contract, on settlement, recomputes the statement digest from the arguments, checks the settlement predicate (Section 10), records the nullifiers, advances the root, and moves any transparent funds, in that order.

7.3 The concrete operations

Op	$\lvert\mathscr{I}\rvert$	$\lvert\mathscr{O}\rvert$	What it does
Shield	0	1	A public deposit a becomes one note; adds a leaf. No input, so no nullifier.
Transfer	1	1	One note becomes one note of equal value for a new owner; adds a leaf.
Split	1	2	One note becomes two, with $n.a = n_1.a + n_2.a$; adds two leaves at i and $i+1$.
Merge	2	1	Two notes become one, with distinct nullifiers; adds a leaf.
Withdraw	1	0	One note becomes a public payout a to $\mathrm{fe}(\text{recipient})$; adds no leaf.

Two cases deserve a note. Split adds two leaves in one operation, so its circuit chains two insertions through an intermediate root: it inserts the first leaf at i to get a middle root, then inserts the second at $i+1$ to get R_{new}, and binds only the endpoints publicly. Withdraw removes value, so it has no output note and binds no new root; instead it binds a hash of the recipient address into the proof, which fixes where the funds go and stops a relayer from redirecting them.

7.4 Extensibility

The general relation admits any $|\mathscr{I}|, |\mathscr{O}| \geq 0$. New shapes, such as a k-input consolidation, an m-output batch payment, or an operation that spends and shields in one step, fit the same relation, the same tree, the same nullifier rule, and the same settlement layer. Adding one requires only a new circuit and a new operation code; nothing else in this specification changes.

8 Public Inputs and the Statement

8.1 Encoding

Every operation exposes a short list of public inputs and nothing else. Integers are encoded by $\mathrm{feUint}(\cdot)$ as 32-byte big-endian field elements; principals by $\mathrm{fe}(\cdot)$ (Definition 4.3); commitments, roots, and nullifiers are already field elements and are taken in their 32-byte big-endian form.

8.2 Statement digest

Definition 8.1 (Statement digest)**.** For public-input field elements $f_0, \ldots, f_{m-1}$ in the circuit's declaration order,

$$h = \mathrm{keccak256}\big(f_0 \,\|\, f_1 \,\|\, \cdots \,\|\, f_{m-1}\big).$$

The digest is the single value that ties a proof to one exact operation. The prover feeds these fields to the circuit; the verifier binds them; the contract recomputes h from the call arguments and checks it against what was verified. Because h depends on every field, changing any argument after proving yields a different h that matches nothing.

8.3 Why the components must agree

The circuit, the client, the two contracts, and the external verifier all hash the same bytes in the same order. If any of them disagreed by a single byte, valid proofs would be rejected and the protocol would not function. For that reason the field list per operation is treated as part of the specification rather than an implementation choice, and there is a single canonical encoder that every component mirrors.

Op	Public inputs (declaration order; bracketed field is SIP-10 only)
Shield	`op, commitment, owner_commitment, amount, old_root, new_root, leaf_index, [asset_id], version`
Transfer	`op, nullifier, new_commitment, new_owner_commitment, merkle_root, new_root, leaf_index, [asset_id], version`
Split	`op, nullifier, commitment_1, owner_commitment_1, commitment_2, owner_commitment_2, merkle_root, new_root, leaf_index, [asset_id], version`
Merge	`op, nullifier_1, nullifier_2, commitment, owner_commitment, merkle_root, new_root, leaf_index, [asset_id], version`
Withdraw	`op, nullifier, amount, recipient_hash, merkle_root, [asset_id], version`

Example 8.1 (Shield digest)**.** A native shield hashes eight elements in order: $h = \text{keccak256}\big(\text{feUint}(1) \,\|\, C \,\|\, oc \,\|\, \text{feUint}(a) \,\|\, R$
The contract rebuilds this exact concatenation from the call and compares.

Arities per operation are in Appendix B.

9 Tree-Transition Binding

9.1 The problem it closes

If a proof does not fix the new root, an operation can advertise any new root it likes. A shielded pool that accepts an unproven root can be driven to treat a fabricated tree as real, which is a direct route to draining funds. The binding below is what removes that freedom, and it is the property that defines circuit version 2.

9.2 The bound transition

Proposition 9.1 (Bound transition)**.** *If a leaf-adding proof verifies with public old root R_{old}, new root R_{new}, appended leaf ℓ, and index i, then by Definition 5.2 $R_{\text{new}} = \text{root}(\ell, \mathbf{b}(i), \mathbf{s})$ over the same siblings $\mathbf{s}$ for which $\text{root}(\mathsf{EMPTY}, \mathbf{b}(i), \mathbf{s}) = R_{\text{old}}$. Hence R_{new} is exactly the root after inserting ℓ at empty slot i, and no other value admits a valid proof.*

9.3 The on-chain slot check

The circuit fixes that the new root is correct *for some* empty slot i. The contract closes the loop by asserting that the slot the registry actually assigns to the new leaf equals the proof-bound i (otherwise it reverts). Together, the in-circuit relation and the on-chain slot check mean the caller cannot substitute a root or a position. Since R_{old}, R_{new}, and i are all in the statement digest (Definition 8.1), none of them can be altered after proving.

10 Settlement and Verification Backends

10.1 The settlement predicate

Definition 10.1 (Settlement predicate)**.** An operation settles on chain only if $\mathsf{Accept}(h)$ holds for its statement digest h, where Accept attests that a proof with public-input digest h was verified under the registered verification key and version v.

Everything above is independent of how Accept is realized. This is deliberate: the note scheme, the tree, the nullifiers, the operations, and the encoding do not change when the verification method changes. Only the implementation of Accept and the trust it carries change. The realizations below are listed in increasing order of trust minimization.

10.2 Delegated verification (current)

The proof system in use needs elliptic-curve pairings to verify, which the base chain does not provide, so verification is delegated and only its result is checked on chain. The procedure:

1. The prover submits the proof and its public inputs to an external verification network.
2. The network verifies the proof against the registered verification key and records the verified statement, whose digest is h.
3. The network batches many verified statements into one Merkle tree and exposes its aggregation root ρ.
4. An authorized publisher posts ρ to the chain.
5. To settle, the contract recomputes h from the operation arguments, and $\mathsf{Accept}(h)$ is the check that the leaf for h is included under a published ρ, a Merkle-path comparison that is cheap on chain.

This backend adds two trust assumptions: that the external network verifies honestly, and that the publisher does not post a root over an unverified statement. The current system keeps the publisher set small and separate from the deployer key, but the assumption is real. The security review tracks it as finding H-1.

Instantiation. The external network is zkVerify; the publisher posts ρ via `submit-aggregation`; the on-chain inclusion check is a native Merkle-proof verification.

10.3 Federated attestation

The publisher trust above can be reduced without any new chain primitive. Let N independent operators each verify the proof off chain and sign the statement (or the aggregation root). Then $\mathsf{Accept}(h)$ is the check of at least M valid signatures from the N, using the chain's native signature verification. Trust moves from a single publisher to an honest majority of an independent set, and the operators are publicly re-verifiable, so a bad attestation is detectable. This is the planned next step, and it pairs naturally with an in-house verifier set.

10.4 Native pairing-based verification

If the chain gains a pairing operation, as several chains already have, the proof can be verified in the contract directly. A pairing-based system such as Groth16 verifies with a single pairing-product check, so $\mathsf{Accept}(h)$ becomes that check evaluated on chain, with no external party at all. Moving to this backend is a change of proof system behind the same circuits and the same statement, supported by the version tag.

10.5 Native hash-based verification

Pairing-free verification is possible with a hash-based proof system. A verifier for a FRI or STARK argument over a small field needs only hashing and field arithmetic that fit the chain's native integer width, so $\mathsf{Accept}(h)$ is that verifier run on chain, again with no external party. It is the only fully native route that needs no new cryptographic primitive from the chain, at the cost of on-chain verification work, which recursion can reduce.

10.6 What stays invariant

Across all four backends the statement digest h is the same function of the same public inputs, so an operation with any altered argument yields a different h and fails Accept under every backend. The migration path is therefore a change of one component, not a redesign.

11 Protocol State and Invariants

11.1 Registry state

The registry is the single source of on-chain state. It holds the commitment tree and its current root, the set of roots that have been published and are still active, the append-only nullifier set $\mathcal{N}$, the per-asset shielded totals, the roles and the authorized-caller allowlist, and a lifecycle state machine. Every protected write is gated by the identity of the calling contract, so even the owner cannot write commitments, nullifiers, notes, or fees directly.

11.2 Note lifecycle

A note moves through a small state machine: it is created by a leaf-adding operation, it is unspent while its nullifier is absent from $\mathcal{N}$, and it becomes spent the moment its nullifier is recorded. The transition is one-way, and a spent note can never transition again, which is a second independent guard on top of the nullifier set.

11.3 Conservation

Proposition 11.1 (Conservation)**.** *For each asset A, let T_A be the recorded shielded total and B_A the pool's actual balance. Shield increases both by the deposit and withdraw decreases both by the payout. The pool*

checks the observed token balance delta against the claimed amount before it changes any state, so $B_A = T_A$ holds after every operation.

Checking the balance delta rather than trusting a token's reported amount is what defends against a misbehaving asset, such as one that takes a fee on transfer or rebases. Because the check runs before state changes, a pool can never pay out more of an asset than has been shielded.

11.4 Double-spend, isolation, and emergency controls

Proposition 11.2 (No double-spend)**.** *$\mathcal{N}$ is append-only and a repeat registration reverts. A note yields a fixed nullifier, so it can be spent at most once.*

Proposition 11.3 (Asset isolation)**.** *A commitment is a function of its asset identifier α, so a proof for asset A cannot satisfy the relation for a different asset B. Assets share the tree without being interchangeable.*

For incident response the lifecycle state machine composes with per-contract freezes and per-operation switches, so specific roots, verification keys, notes, or whole operations can be halted without touching the rest of the system.

12 Fees

Per asset, with rate bps in basis points and a flat term ϕ, the fee on a public amount a is

$$\text{fee}(a) = \phi + \left\lfloor \frac{a \cdot \text{bps}}{10000} \right\rfloor, \qquad \text{fee}(a) < a.$$

Only shield and withdraw expose a public amount, so only they can carry a percentage. Operations over hidden amounts admit only a flat term, because a percentage would need the amount in the clear. The fee model is also designed for privacy: the shield fee is folded into the public deposit, and the withdraw fee is paid out by the pool contract rather than by the user, so a fee payment cannot be linked to an individual.

Instantiation. Shield 25 bps, withdraw 30 bps, and $\phi = 0$ for the hidden-amount operations.

13 Security Model and Properties

13.1 Threat model and assumptions

The adversary may read all chain data, submit malformed or replayed operations, deploy a misbehaving token, act as a malicious relayer, and operate a curious indexer. The guarantees below hold under two assumptions: the primitives (H, the proof system Π, and keccak256) are secure, and the chosen Accept backend behaves as specified (Section 10). The delegated backend adds trust in the external verifier and the publisher; the later backends remove it.

13.2 Properties

- **Soundness.** A settled operation has a verified proof whose digest the contract re-derived, so its relation holds, value is conserved, and, for a leaf-adding operation, the tree transition is bound (Proposition 9.1).

- **No parameter forgery.** Every parameter is inside the statement digest, so no argument can be changed after proving.
- **No double-spend or replay.** The append-only nullifier set, the note-state machine, and the per-settlement records each independently reject a repeat.
- **Conservation and isolation.** Proposition 11.1 and asset isolation bound payouts per asset and prevent cross-asset spends.
- **Confidentiality.** On-chain data reveals no amount, owner, or linkage; note contents are encrypted to the owner and recovered by trial-decryption.
- **Sender privacy.** A relayer submits the transaction and cannot alter it, so the chain never sees the user. This is a liveness dependency only, and a decentralized relayer set removes the chokepoint.

A Current Instantiation

Parameter	Value
$\mathbb{F}$	BN254 scalar field, $\lvert\mathbb{F}\rvert = 21888242871839275222246405745257275088548364400416034343698204186575808$
Π	UltraHonk over BN254 (Barretenberg)
E	Grumpkin (embedded curve of BN254), generator G; $sk \in [1, 2^{240}]$
H	Poseidon: H_2 for leaf, owner, and nullifier; H_4 for the commitment
keccak256	Keccak-256 over big-endian field elements
d, EMPTY, v	20, 0, 2
Fees	shield 25 bps, withdraw 30 bps, others 0
Accept	delegated (zkVerify aggregation inclusion)

B Public-Input Arities (version 2)

Operation	Native STX	SIP-10
Shield	8	9
Transfer	8	9
Split	10	11
Merge	9	10
Withdraw	6	7

The SIP-10 family adds one `asset_id` field element immediately before `version`, and is otherwise identical in structure.

Companion document: the White Paper covers motivation, use cases, and roadmap. The internal security review is in the `audits/` directory of the project repository.